PROJECT AND TEAM INFORMATION

Project Title

Detective Mystery: The Hidden Truth

Student / Team Information

Team Name : **Team ID :**	**Truth Seekers** *T152*
Team Member 1 *Team Lead*	**Badhani, Karan** Student ID: 2510360037 *karanbadhani9770@gmail.com* 
Team Member 2	**Giri Goswami, Suraj** Student ID: 2510012585 *2510012585@geu.ac.in* 
Team Member 3	**Singal, Khushi** Student ID: 2510360039 *singalk20@gmail.com* 
Team Member 4	**Arora, Tanmay** Student ID: 2510610178 *2510610178@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

During our study of C++, OOP and Data Structures, we noticed that most concepts are practiced through short and separate programs. This is useful for learning their basic working, but it does not always show how several concepts can be combined while building one application. We wanted our project to give us that experience.
For this reason, we chose to build "The Hidden Truth", a detective mystery game. The player takes the role of an investigator and works through a fictional case. Instead of simply moving through a fixed story, the player visits locations, speaks with suspects, discovers clues and studies the evidence before making a final decision.
The idea also gives us a practical reason to use the topics covered in our course. Locations and routes can form a graph, previous investigation steps can be recorded using a stack, and pending tasks can be managed through a queue. Clues and evidence can be stored in a way that makes them easy to search and retrieve. The characters, locations and investigation records can also be organized as C++ objects.
Our main motivation is to learn these concepts by actually using them. Instead of writing an unrelated program for every topic, we want to see how OOP and DSA work together while building and testing one complete project.

State of the Art / Current Solution

OOP and Data Structures are normally introduced through lectures, laboratory work and individual coding problems. A student may create a stack in one program, implement a queue in another and study graphs through a separate example. This approach explains the operations clearly, but the connection between these topics becomes more visible when they are used together.
Detective games provide a useful setting for such an implementation because an investigation already contains connected places, stored information, previous actions and different records that need to be examined. However, the main purpose of a normal mystery game is entertainment, not the demonstration of programming concepts.
In our project, these programming concepts become part of the game's working logic. The investigation map is linked with graph concepts, collected information is stored and retrieved during gameplay, and previous actions can be maintained for backtracking. Searching and sorting also become useful when the player has several clues or suspects to examine.
The purpose is therefore not to build a collection of separate DSA demonstrations. We are using one mystery case as a common problem in which each selected concept performs a specific job.

Project Goals and Milestones

Our goal is to complete a playable C++ mystery in which the investigation itself depends on the OOP and DSA concepts studied in the course. The project should allow us to explain not only what the game does, but also why a particular programming concept was chosen for a feature.

Major Goals

1. *Prepare one complete fictional case with locations, suspects, clues and evidence.*
2. *Represent the main parts of the game through suitable C++ classes and objects.*
3. *Use encapsulation, abstraction, inheritance and polymorphism where they fit the class design.*
4. *Build the location system using graph concepts and use suitable structures for investigation records and tasks.*
5. *Allow the player to collect, store, search and review information found during the case.*
6. *Use searching and sorting where the investigation requires records to be located or arranged.*
7. *Combine these modules into a working prototype and test the complete case from beginning to final deduction.*

Initial Milestones

Milestone 1: *Write the case and decide the suspects, locations, clues and correct solution.*
Milestone 2: *Prepare the class design and define the responsibility of each major class.*
Milestone 3: *Match the required game operations with suitable data structures and algorithms.*
Milestone 4: *Code the basic player, location, suspect, clue and evidence components.*
Milestone 5: *Connect navigation, clue collection and investigation records with the main game flow.*
Milestone 6: *Test the modules together and correct problems in the logic or movement of data.*
Milestone 7: *Finish the case, test different player choices and prepare the prototype for demonstration.*

Project Approach

We plan to build the game in small working parts rather than writing the whole program at once. Our first task is to finish the mystery itself. This includes deciding what happened in the case, which characters are involved, where clues are located and what evidence is required to reach the correct conclusion.

Once the case is fixed, we will prepare the C++ class structure. Player, Suspect, Location, Clue and Evidence will handle the information directly related to those entities. Case and manager classes can then coordinate the investigation and the overall game state. Keeping these responsibilities separate should make the program easier for us to test and modify.

The locations in the mystery will be stored as connected points of the game map. Graph operations will handle these connections and determine where the player is allowed to travel. When the player moves through the investigation, earlier steps can be placed on a stack so that a previous step can be revisited when required. Tasks that are waiting to be investigated can be kept in a queue and handled in their stored order.

Clues and evidence will have identifiers along with the information required by the case. A hash-based structure can associate an identifier with its record, reducing the work needed to locate a stored item. Searching will be used when a particular record is requested. Sorting will be applied only where an ordered view is useful, such as arranging records by priority or suspects by a selected value.

After the individual parts are working, we will connect them and play through the case ourselves. This stage will help us find missing links between clues, incorrect routes, invalid choices and problems in the program logic. Visual improvements will be considered after the investigation can be completed correctly from start to finish.

System Architecture (High-Level Diagram)

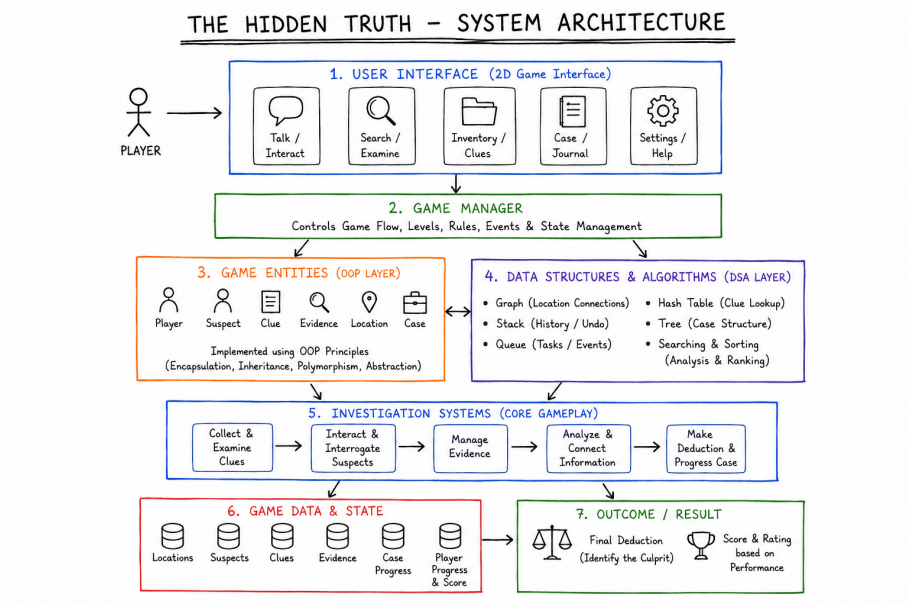

Figure 1: High-level system architecture of The Hidden Truth

Project Outcome / Deliverables

The first completed version of "The Hidden Truth" is planned as a small but complete investigation rather than a large game with many unfinished features. A player should be able to begin the case, travel through the available locations, obtain information from suspects, collect the required clues and reach the deduction stage.

For the project demonstration, our team intends to present:

- *one complete mystery with a defined beginning, investigation path and solution;*
- *a set of connected locations through which the player carries out the investigation;*
- *suspects with information that becomes useful at different stages of the case;*
- *clues and evidence records that can be collected and reviewed;*
- *C++ classes responsible for the main characters, records and game operations;*
- *graph-based connections between the locations;*
- *stored investigation history that supports returning to an earlier step when needed;*
- *an ordered set of pending investigation tasks;*
- *retrieval, searching and sorting operations used on investigation data; and*
- *a deduction stage in which the available evidence is checked before the player makes the final choice.*

Along with playing the case, we will be able to show the relevant part of the program behind each major operation. This will make it possible to explain how the data structures and the C++ class design contribute to the working of the game.

Assumptions

For Phase I, we are treating this project as a course prototype. We are not trying to create a large commercial detective game. Keeping the first case small gives us enough time to check the investigation properly and verify that the required C++ and DSA operations behave as expected.
The mystery used for the prototype will be fictional. Its characters, locations, clue descriptions and evidence will be prepared for this project. Only the locations required by the case will be included in the initial map, and the number of suspects will remain manageable during development.
The program is expected to run on a regular computer with a suitable C++ development setup. Our priority is the correctness of the case logic, navigation, stored records and data-structure operations. Advanced animation or detailed graphics are outside the essential scope of the first prototype. Such interface improvements can be considered later without changing the basic investigation design.

References

1. *SFML Documentation:* `https://www.sfml-dev.org/documentation/`
2. *C++ Reference:* `https://en.cppreference.com/`
3. *GeeksforGeeks – Data Structures:* `https://www.geeksforgeeks.org/data-structures/`
4. *Microsoft Learn – C++ Documentation:* `https://learn.microsoft.com/en-us/cpp/`
5. *Programiz – Data Structures and Algorithms:* `https://www.programiz.com/dsa`